

Reatividade

A programação reativa, que teve seu nome derivado do framework React, é um paradigma de programação criado inicialmente pela Microsoft e que vem ganhando aceitação no desenvolvimento de interfaces de usuário. Para mobile e desktop os frameworks reativos em evidência são [React Native](#) e [Flutter](#), enquanto para a Web temos [React](#) e [Vue.js](#).

É orientado a fluxo de dados e propagações de estados, onde esses fluxos (também chamados de *streams*) são em grande parte assíncronos, ou seja, as operações são independentes umas das outras e não precisam ser executadas em uma sequência específica.

Todas as ações são transmitidas e detectadas por um fluxo de dados, como eventos, mensagens, chamadas e até mesmo as falhas. Desta forma, aplicações reativas são constituídas por reações a alterações nestes fluxos de dados.

O paradigma reativo possui alguns conceitos vantajosos como a assincronia, utilização de processos não-bloqueantes e orientação a mensagens e eventos. Além disso, aplicações reativas tendem a ser mais escaláveis uma vez que todos os processos são fundamentados em cima de eventos não-bloqueantes. A execução de tarefas bloqueantes geralmente tem a tendência de criar bloqueios de recursos computacionais (como memória e processador), o que pode atrapalhar na escalabilidade da aplicação.

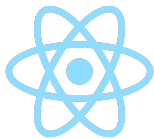
Nova forma de construção do frontend

Antes da utilização de programação reativa, uma página Web criada, preenchida com algumas propriedades, precisava ser inteiramente recarregada caso alguma dessas propriedades fosse alterada. Era preciso de alguma forma monitorar mudanças nas propriedades e invocar a atualização da interface de usuário de acordo.

Isso é muito semelhante à renderização de jogos 3D, onde a *Graphics Processing Unit* (GPU) repinta a tela inteira a cada frame. Porém a geração de código HTML não conta com a aceleração GPU.

Com a utilização do React ocorre uma otimização nessa geração de código, uma vez que aplica somente às diferenças na DOM. Isso acaba gerando uma “economia” de processamento uma vez que a página “reage” automaticamente às mudanças.

E, se levarmos em consideração que as páginas Web são desenvolvidas segundo uma hierarquia de componentes, tem-se que a mudança de uma propriedade afeta uma porção muito pequena da página, e a detecção de mudanças na DOM precisará verificar apenas os componentes que dependem dela.



O Manifesto Reativo

Criado em 2014 por um grupo de desenvolvedores, o [Manifesto Reativo](#) define características que fazem com que uma aplicação seja considerada reativa. De acordo com a versão mais recente, existem 4 pilares que orientam como os sistemas reativos devem ser:

- **Responsividade (responsiveness)** - aplicações reativas precisam oferecer um feedback rápido para as ações desempenhadas pelo usuário. Para ser responsivo, um sistema deve oferecer interações ricas em tempo real com os usuários, estabelecendo uma margem de tolerância que garantam uma qualidade de serviço. Esse comportamento transmite confiança ao usuário final, além de simplificar o tratamento de erros.
- **Resiliência (resilience)** - aplicações reativas devem reagir e se recuperar de falhas de software, hardware e conectividade, além de serem capazes de “lidar” com o componente que apresenta o comportamento incorreto. Para isso, existem algumas estratégias que podem ser adotadas, como mecanismos de replicação (onde uma aplicação possui uma cópia principal funcional e réplicas “adormecidas” que podem ser instantaneamente acionadas caso a cópia principal comece a falhar), mecanismos de contenção (como uma aplicação que é capaz de “segurar” as requisições realizadas para que sejam processadas novamente quando ela se recuperar de um episódio de falha), mecanismos de isolamento (como em uma aplicação que exibe um feedback de indisponibilidade de uma funcionalidade quando esta começa a apresentar um índice alto de falhas ao invés de permitir aos usuários continuar acessando o processo que está com problemas) e mecanismos de delegação (quando uma aplicação consegue utilizar mecanismos de fallback para ações defeituosas). Qualquer sistema que não esteja resiliente naturalmente não será responsivo após uma falha.
- **Elasticidade (elasticity)** - a elasticidade de um sistema é verificada quando ele se mantém responsivo e resiliente mesmo com altas variações de carga, sendo que esse processo de elasticidade precisa ser muito rápido e automático. O sistema irá reagir a grandes cargas, aumentando ou diminuindo os recursos alocados para servir estes mesmos acessos.
- **Guiado por mensagens (message driven)** - uma arquitetura orientada por mensagens é a base das aplicações reativas. Um sistema orientado por mensagem reage aos eventos em vez de compor aplicações por múltiplas threads síncronas, onde os sistemas são compostos de gerenciador de eventos assíncronos e não-bloqueantes. Isso quer dizer que, em uma aplicação reativa, ao invés de você ter chamadas explícitas entre componentes (um controller e um repository, por exemplo), você tem na verdade mensagens trocadas de maneira assíncrona entre estes dois componentes. Estas mensagens geralmente são trafegadas entre um meio transparente entre os componentes, como um barramento de eventos (event bus), e acabam constituindo justamente os data streams.